

Three sequencers as PureScript live-coding higher structures

Date: 2026-05-09 **Author:** Claude (with Andrew) **Status:** research / interrogation, not a port commitment

Context

Andrew owns three eurorack sequencers/modulation sources whose feature sets are interesting candidates for porting into purerl-tidal as **higher-structure cell vocabulary**:

1. **Make Noise René 2** — 4×4 Cartesian step sequencer with three channels, snake/cartesian walkers, 64-state Z-axis memory.
2. **ALM Pamela's NEW Workout** — 8-channel programmable clocked modulation source with rich per-output waveform/Euclidean/random parameters.
3. **Noise Engineering Mimetic Digitalis** — 4-CV-output recallable step sequencer with a 5-direction navigation vocabulary.

The framing Andrew gave: *"Not that I'd necessarily want copies of these modules that I already have, it's more to interrogate the possibilities and also, yes to see if those are attractive higher structures to work with in a live-coding environment."*

So this is **not a port plan**. It's a thinking artefact answering:

- Which of these three has structure that genuinely lifts above what mini-notation already gives us?
- What would the cleanest PureScript surface look like?
- What's reachable on the rig (FH-2 + FHX-8GT, ES-9 + ES-5 + ESX-8CV, driven by cv-router and link-spike), and what isn't?
- If we did do this work, in what order, and what would we skip?

Companion to (and continuation of) the earlier [reference_modulation_module_research](#) memory from 2026-04-25, which covered six modules at lower depth.

TL;DR — verdict per module

Pamela's NEW Workout: yes, port. Pam's data model is *already* a typed parameter-bag specification for clocked modulators. It maps cleanly onto a typed mod-matrix language extension we don't currently have, and addresses a real gap: we have nothing for authoring 8 simultaneous shaped CV streams. Highest language value of the three; lowest implementation cost. Aligns with the "Metropolix mod matrix as North Star" call from the prior research.

Mimetic Digitalis: yes, with the inversion. The hardware navigates a 16-step bank via *patched gates* — N/X/Y/R/O trigger inputs. In a live-coding language, we make the **navigator itself a Pattern of typed actions**, which is strictly more expressive than the hardware. The 5-verb action set is small enough to be a genuine vocabulary (not a flag set), composes with mini-notation and with everything we already have. Modest scope, high return.

René 2: extract one principle, skip the rest. The two-clock Cartesian walker is the genuinely novel idea — small clock-ratio changes produce large pattern shifts, which is hard to express in mini-notation. Lift that as a Pattern combinator. The rest of the module (Z-axis state machine, mesh-paste, latch, touch UI) is hardware-UX whose *value comes from the gestural interface*, not the algorithmic shape. Porting whole would be expensive and lose what makes the module work.

Metropolix (added pass): don't port standalone — let it emerge. Metropolix is essentially "Pam plus a typed per-stage algebra plus a runtime mod-graph with self-modulation." Each ingredient is a natural *evolution* of the Pam port; building Metropolix first means building all of those pieces speculatively. Build Pam, use it, then graduate toward Metropolix's distinctive features (mod-matrix-as-runtime-graph, MOD-lanes-targeting-each-other, per-stage (`pulse_count`, `gate_type`, `ratchet`, `prob`, `accum`, `slide`, `skip`, `cv`) algebra) only as Pam's limits make them feel necessary. The hardware Metropolix is best understood as the **target shape** of the language extension, not a separate port.

Marbles' DEJA VU: yes, as a single Pattern combinator. The whole Marbles module is too sprawling, but DEJA VU's "store a seed, not voltages" mechanic is a beautifully small idea that **lives on top of any random source**. Lift it as a parametric combinator applicable to any **Pattern a** whose semantics involve randomness: `dejaVu { length, lockProb, shuffleProb } pat`. About a day's work, and it fills a genuine gap (mini-notation has no clean way to author "random-but-recoverable").

Elektron Digitakt-style — yes, this might be the strongest model of all six. The Digitakt's three core mechanisms — **parameter locks** (sparse per-step parameter overrides over track defaults), **pattern pages** (>16-step patterns played as cycling pages), and **conditional trigs** (PRE / NEI / 1ST / A:B / X%) — together form the most directly *live-coding-shaped* model in this analysis. The data structure is already declarative: track defaults plus a sparse **Map StepIndex (Map Param Value)**. Pages are just **Pattern (Pattern Step)**. Each mechanism is its own axis, all three compose, and they map to clean PureScript record/Map types with no runtime metaprogramming. **Worth seriously considering ahead of Pam, or as Pam's per-step layer.**

Method

Read all three current manuals end-to-end:

- [Make Noise René2 Manual.pdf](#) (33 pp.)
- [ALM pamelas-new-workout.pdf](#) (25 pp., manual v0.42, firmware 199/200)
- [Mimetic Digitalis – Noise Engineering Documentation.html](#)

Cross-referenced against the rig capability map (cv-router buses, ES-5/ESX-8CV/ESX-8GT/FH-2/FHX-8GT routing) and against the existing purerl-tidal vocabulary (mini-notation, Branched, fork/merge, Tidal.Cell.Prelude). Considered each module under three lenses:

1. **Hardware feature inventory** — what does the module actually do.
2. **Live-coding ergonomics** — does the *shape* of this module make sense as a typed cell vocabulary, or is its value tied to the physical UI?
3. **Rig coverage** — given FH-2 (16 gates via FHX-8GT chain), ES-9 (16 buses via cv-router), ES-5 (6 + 8 gates via ESX-8GT), and ESX-8CV (8 audio-rate CVs via Silent Way encoding), what's addressable and at what latency.

What "porting" means here (it's not a copy)

A core insight applies uniformly to all three modules:

Hardware modules navigate stored state via patched **gates** — a small vocabulary (rising edges, gate highs/lows). The state-stores themselves are rich, but the navigation surface is impoverished by physical-jack reality. In a live-coding language, we replace "patched gate inputs" with **a Pattern of typed navigation actions**. The action set can be larger, the actions can be authored in mini-notation, and the navigator is itself composable with **slow, fast, rev, jux**, etc.

This is the inversion that makes any of this work worth doing. Without it, we'd just be reimplementing things Andrew already owns. With it, we get strictly more expressive instruments than the originals — at the cost of giving up the gestural-touch UX, which we replace with keyboard-authored cells.

The general shape of each port:

```
data Sequencer state nav out = Sequencer
  { state :: state                -- the stored vocabulary
  , nav   :: nav -> state -> state -- one navigation step
  , read  :: state -> out         -- what comes out at current pos
  }

-- And the running form, where the navigator is itself a Pattern:
runSeq :: Sequencer s n o -> Pattern n -> Pattern o
```

Per-module:

- **René 2:** `s = (Grid (Cell × Cell × Cell), (col,row)), n = TwoClockTick, o = (CV_x, Gate_x, CV_y, Gate_y, CV_c, Gate_c).`
- **Pam's:** `s = 8 × Recipe + ClockPhase, n = ClockTick, o = 8 × Voltage.`
- **Mimetic:** `s = 16 × Vec4 + Position, n = NavAction (5 verbs), o = Vec4.`

Rig coverage map

For grounding, what's reachable from the rig:

CV destinations (post-cv-router and Silent Way)

Source	Channels	Update	Slew	Notes
ES-9 (cv-router)	16 buses	sample-accurate at 48 kHz	none on hardware; software slew per-bus is cheap	Buses 8–15 → ES-9 panel jacks 1–8
ESX-8CV (via ES-5)	8 CVs	~750 Hz (Silent Way's 24-bit PCM rate)	none	-10 V to +10 V, 1 V/oct

Source	Channels	Update	Slew	Notes
FH-2 MCV outs	8	MIDI-rate	optional via firmware portamento	Has glide; used for melodic CV

Total ~32 CV destinations — overkill for any of these three modules (René peaks at 3, Pam at 8, Mimetic at 4).

Gate/trigger destinations

Source	Channels	Notes
ES-5 + ESX-8GT	6 + 8 = 14	Encoded via Silent Way; cv-router emits it
FH-2 + FHX-8GT	8 + 8 = 16	MIDI-velocity-as-gate-amplitude possible too

Total ~30 gates. Plenty of headroom.

What's hard / impossible on the rig

- **Per-cell glide on a CV bus** (René's GLIDE page): cv-router emits step-changes; software-side ramp would need to be added per-bus. Cheap (a single linear ramp in the per-bus sample loop) but not there yet. **Cost to add: small.** A per-bus `slewMs :: Number` configuration plus linear interpolation.
- **Audio-rate CV** (e.g. Pam's "envelope" wave at fast modifier values): ES-9 is sample-rate, no problem. ESX-8CV is ~750 Hz, fine for shaped LFOs but not envelopes-as-attack-stages.
- **State recall via touch interface:** irrelevant — we're authoring in code, not in panels.

Sample-accuracy of clocking

- cv-router clocks each bus from a single `Pattern (Time, Voltage)` stream produced by the BEAM scheduler.
- link-spike provides Ableton Link tempo on `:20808` and emits `OSC /cv/trig/at` with target wall-clock for sample-accurate gate timing.
- Per-voice supervisor (current refactor on `per-voice-supervisors`) schedules events per voice at sub-millisecond precision.

So everything described below runs against a single temporal substrate, which is one of the biggest advantages over the modular implementations (where clock distribution is its own problem).

Module 1 — Pamela's NEW Workout

Verdict: port. Highest value, lowest cost.

Hardware feature inventory

8 outputs, each independently configurable. Per-output recipe:

Parameter	Type / Range	Notes
Modifier	$\div 512 \dots \div 1$, $\times 1 \dots \times 48$, plus <i>On</i> / <i>Off</i> / <i>PulseOnStart</i> / <i>PulseOnStop</i> utilities, plus dotted/triplet decimals	Clock divisor or multiplier of master BPM
Wave	<i>Gate</i> <i>Triangle</i> <i>Sine</i> <i>Envelope</i> <i>Random</i>	One full cycle covers one step
Level	0-100 % of 5 V max	
Offset	0-100 % of 5 V	Vertical offset; clipped at 5 V (firmware ≥ 189)
Width	0-100 %, meaning depends on Wave	Gate $\rightarrow$ duty; Tri/Sine $\rightarrow$ skew; Env $\rightarrow$ release; Random $\rightarrow$ no-op
Phase	0-100 %	Start point on chosen waveform
Delay	0-100 % of step time	Offset before waveform begins (clipped at end)
DelayDivisor	<i>1, 2, 3, ...</i>	Which steps get delayed (every Nth); divisor of 2 = swing
Slop	0-100 %	Timing humanisation (jitter)
EStep	int	Euclidean: total step count
ETrig	int < EStep	Euclidean: number of hits
ERot	int	Euclidean: rotation
RSkip	0-100 %	Per-step random skip probability
Loop	beats (or "free")	Resets random-seed and Euclidean phase every N beats — the Loop param is the conceptual key for <i>repeatability of randomness</i>

Plus 2 CV inputs (CV1 0-5 V, CV2 -5 to +5 V), each assignable to any output parameter with per-assignment **attenuation** and **offset**. Multiple parameters can share one CV.

Per-output banks: 26 letter banks $\times$ 8 slots, plus full-bank load/save.

Master clock: 10–300 BPM, syncable via external clock + Run inputs at 1–48 PPQN (recommended 24).

Live-coding ergonomics

Pam's recipe-per-output structure is **already exactly what we want**. Each parameter is an independently-typed value, the parameter set is closed and small, and the patternable axis (CV-assigned params) is explicit. No translation required — just lift to PureScript.

The single most interesting feature for live-coding: the **Loop** parameter. It declares "this output's randomness re-seeds every N beats" — meaning random + Loop = *deterministic-looking-random*, which is exactly what live-coders reach for when they want "evolving but recoverable" textures. mini-notation has no equivalent; once a ? is spoken, that voice forks unpredictably forever.

Proposed PureScript surface

```
module Tidal.Pam where

import Tidal.Pattern.Types (Pattern)

-- Per-output recipe, explicit and patternable
type Lane =
  { modifier  :: Modifier          -- mul/div/special
  , wave      :: Wave              -- closed ADT
  , level     :: Number            -- 0..1
  , offset    :: Number            -- 0..1
  , width     :: Number            -- 0..1
  , phase     :: Number            -- 0..1 cycle
  , delay     :: Number            -- 0..1 of step
  , delayDiv  :: Int               -- 1=every step, 2=swing
  , slop      :: Number            -- 0..1
  , euclid    :: Maybe Euclid
  , rSkip     :: Number            -- 0..1
  , loop      :: Maybe Int         -- beats; Nothing = free
  }

data Modifier
= Mul Int      -- x1 .. x48
| Div Int      -- /1 .. /512
| TripletMul Int | TripletDiv Int
| DottedMul Int | DottedDiv Int
| AlwaysOn | AlwaysOff
| PulseStart | PulseStop

data Wave = Gate | Triangle | Sine | Envelope | Random

type Euclid = { steps :: Int, trigs :: Int, rot :: Int }

-- Default recipe: free-running gate at one-per-beat
defaultLane :: Lane
defaultLane =
  { modifier: Mul 1, wave: Gate, level: 1.0, offset: 0.0
  , width: 0.5, phase: 0.0, delay: 0.0, delayDiv: 1
  , slop: 0.0, euclid: Nothing, rSkip: 0.0, loop: Nothing
  }

-- A bank of 8 lanes plus a mod-matrix of patternable parameters
type Pam =
  { lanes :: Vec8 Lane
  , mods  :: Array ParamMod
  }
```

```
-- Each ParamMod ties a Pattern to one (lane, param) destination
data Param = PLevel | POffset | PWidth | PPhase | PDelay | PSlop
           | PESTeps | PETrigs | PERot | PRSkip | PLoop
type ParamMod = { lane :: LaneIdx, param :: Param, pat :: Pattern Number }

-- Run a Pam configuration as a Pattern of 8-channel CV emissions
runPam :: Pam -> Pattern (Vec8 Voltage)
```

Example cells

```
-- A four-on-the-floor kick gate plus a tri LFO at /4 modulating filter
example1 :: Pam
example1 = pam (defaultLane { modifier = Mul 1, wave = Gate, width = 0.1 })
             (defaultLane { modifier = Div 4, wave = Triangle })
             (defaultLane { modifier = AlwaysOff })
             -- ... 5 more

-- Euclidean 5/8 hat with re-seeding every 4 beats
hat :: Lane
hat = defaultLane
    { modifier = Mul 8
    , wave      = Gate
    , width     = 0.25
    , euclid    = Just { steps: 8, trigs: 5, rot: 0 }
    , rSkip     = 0.15
    , loop      = Just 4
    }

-- Mod-matrix: the bass-filter cutoff (lane 1's level)
-- itself follows a slow pattern of values
melodic :: Pam
melodic = (pam ..) `withMods`
    [ { lane: l1, param: PLevel
      , pat: slow 8 (mini "0.3 0.5 0.7 0.9 0.7 0.5") } ]
```

The crucial bit: **withMods** lets us pattern-modulate any of the recipe parameters at any rate. This is the **patternable-mod-matrix** the prior memo flagged as the language-extension North Star, with Pam's existing schema as the instantiation.

What's gained vs the hardware

- Mod destinations are the full recipe, not just the two CV inputs Pam exposes physically.
- Recipes are values, not flash states — version-controlled, copyable, composable, hashable into voice cells.
- Loops are explicit Pattern values rather than knob-mediated; the random-seed-with-recoverability story works without combo moves.
- The 8 lanes can be routed to *any* CV destination on the rig, not just the 8 jacks of one Pam.

What's lost

- The OLED display and program-knob immediacy. You can't grab a knob and twist it; you edit a cell and re-cue.
- The 26-bank flash storage doesn't translate (we have git instead; arguably better).
- The CV-input attenuverter knobs — but we replace these with explicit **Pattern Number** values, which are richer.

Estimated implementation cost

Phase 1 (~3-5 hours): the typed Lane + Pam + runPam producing gate/wave events per cycle. Reuses existing Pattern infrastructure (time-keyed event streams). Output is **Pattern Voltage** per lane.

Phase 2 (~2-3 hours): mod-matrix overlay (parametric Patterns mutating recipe fields per query window). Mostly a record-update fold over **mods**.

Phase 3 (~2-3 hours): rig binding — mapping **Vec8 Voltage** to 8 specific bus channels via a **bind-pam** directive analogous to the existing **midi-device** / **bind** lines.

Total: ~10 hours to a working **pam** cell vocabulary. **This one is worth pulling forward.**

Module 2 — Mimetic Digitalis

Verdict: port, with the inversion.

Hardware feature inventory

- 4x4 grid of 16 steps (single position shared across outputs).
- 4 individually-editable CV outputs (0–5 V, unquantised) + 1 trigger output that fires on every step advance.
- Five **navigation triggers**:
 - **N** — next step (full 16-step wrap)
 - **X** — next column in current row
 - **Y** — next row in current column
 - **R** — random step
 - **O** — origin (step 1)
- Three CV inputs (CV-N, CV-X, CV-Y) for continuous-CV step addressing.
- 16 pattern save slots; combo moves for fast save/load.
- Stop/Run; live-record via the encoder while running.
- Operations: **Zero** (silence one or all), **Shred** (random one or all), **PitchShred** (one-octave random), **Slide** (rotate sequence origin).
- Undo to last loaded pattern.

What makes Mimetic interesting for live-coding

The 5-action navigation vocabulary is **small enough to be a real language**, not a flag set. Compare:

- Most step sequencers are a single "next" gate — one verb.
- René 2 has X-clock and Y-clock — two verbs, but they're channels not actions.

- Mimetic has five distinguishable nav verbs, each producing a different motion through 2D space.

In hardware, you're limited to whatever gates you can patch — usually two or three at most. In code, **the navigator is a Pattern of these five actions**, which is much richer:

```
"n n n x  n n y r  n n n x  o n n n"
```

...describes a 16-cycle traversal with a clear gestural shape: walk forward, occasionally jump sideways, occasionally go random, occasionally reset. This is an *expressive sequencing primitive* that mini-notation alone doesn't give us — mini-notation indexes a sequence linearly; this lets you index a *2D bank* via a pattern of typed moves.

Proposed PureScript surface

```
module Tidal.Mimetic where

-- The bank: 16 steps, each a 4-vector of CV values
type Bank = Array (Vec4 Voltage)
-- (length 16; could be enforced with a refined type later)

-- The five navigation actions
data Nav = N | X | Y | R | O

-- Position on the 4x4 grid
type Pos = { row :: Int, col :: Int }

-- Apply one Nav to a position
step :: Nav -> Pos -> Effect Pos      -- Effect because R uses RNG
step N p = pure $ wrapNext p          -- 16-step row-major
step X p = pure $ p { col = (p.col + 1) `mod` 4 }
step Y p = pure $ p { row = (p.row + 1) `mod` 4 }
step R _ = randomPos
step O _ = pure { row: 0, col: 0 }

-- Run: at each event in the nav-pattern, advance position, emit Vec4
mimetic :: Bank -> Pattern Nav -> Pattern (Vec4 Voltage)

-- Convenience: bank constructors
zeros :: Bank
fromValues :: Array (Vec4 Voltage) -> Bank
shred :: Effect Bank                  -- random; returns Effect for seeding
pitchShred :: Effect Bank             -- one-octave random
```

Example cells

```
-- Author the bank as a 4x4 of 4-CV vectors
-- (each CV could drive a different parameter: pitch, gate length,
```

```

-- filter cutoff, resonance, say)
bassBank :: Bank
bassBank = fromValues
  [ v 60 0.5 0.3 0.0, v 0 0 0 0, v 67 0.5 0.7 0.0, v 0 0 0 0
  , v 60 0.8 0.4 0.0, v 64 0.3 0.5 0.5, v 67 0.5 0.6 0.2, v 72 0.5 0.8 0.0
  , v 60 0.5 0.3 0.0, v 0 0 0 0, v 65 0.5 0.5 0.0, v 0 0 0 0
  , v 60 0.8 0.4 0.0, v 64 0.3 0.5 0.5, v 65 0.5 0.6 0.2, v 72 0.5 0.8 0.0
  ]

-- Linear walk
linear :: Pattern (Vec4 Voltage)
linear = mimetic bassBank (mini "n*16")

-- Cartesian-style: alternating X and Y moves, occasional resets
cartesian :: Pattern (Vec4 Voltage)
cartesian = mimetic bassBank (mini "n x n y n x n y n x n y n x o y")

-- Probabilistic walker
walker :: Pattern (Vec4 Voltage)
walker = mimetic bassBank (mini "<n n n>?0.7 <x y>?0.3 r")

```

The navigator-as-pattern is the core idea. Once you have it, you can do things the hardware can't:

- `jux rev (mimetic bank "n*16")` — run the same bank, two voices, one walking forwards, one walking backwards.
- ``every 4 (mimetic bank ("o n*15"))` — once every four cycles, reset to origin and walk linearly; otherwise walk with another pattern.
- Chain banks: `alternate [bank1, bank2] (mimetic _ "n*16")`.

What's gained vs the hardware

- The navigator is itself patternable — way richer than 5 patch cables.
- Banks are values, composable across cells.
- Multiple navigators can drive the same bank simultaneously (impossible in hardware — one position).
- `Shred` and `Zero` are operations on the value, applied at edit time; no live-record needed.

What's lost

- The encoder live-record paradigm. We replace this with re-cuing a cell that has different bank values — slower but version-controlled.
- The combo-move keyboard shortcuts are irrelevant.
- The single-position constraint is *not* lost — we can choose to keep it (one walker per Mimetic cell) or relax it (multiple walkers, one bank). Worth defaulting to the constrained shape (closer to hardware feel) and providing a multi-walker variant as opt-in.

Estimated implementation cost

Phase 1 (~2-3 hours): the typed Bank + Nav ADT + step function + mimetic combinator producing `Pattern (Vec4 Voltage)`. Mostly a fold over the nav pattern.

Phase 2 (~1-2 hours): bank-construction helpers (zeros, shred, pitchShred, slide) — small functions.

Phase 3 (~2 hours): rig binding for 4-CV vector emission to a chosen tuple of buses.

Total: ~6 hours to a working **mimetic** cell vocabulary. **Smaller than Pam, smaller scope, and complementary — they don't overlap. Worth doing both.**

Module 3 — René 2

Verdict: extract one principle, skip the rest.

Hardware feature inventory

Three channels — **X** (red), **Y** (green), **C** (Cartesian, orange). Each X/Y has independent 4x4 grid programming and snake-walks the grid. C inherits position from X.col and Y.row (two-clock cartesian).

Per channel, six **Program Pages** of 16-button programming:

Page	Function	Per location
ACCESS	Skip mask	accessible/skip
GATE	Gate enable	gate/silent
GLIDE	Portamento	glide-from-prev/no
SNAKE	Walk pattern (X/Y only)	one of 16 hardcoded curves
FUN	Three rows: OP, MOD-input behaviour, CV-input behaviour	
QUANT	12-tone scale select + octave range (1-4)	

Plus 16 knobs for the 16 cell CV values per channel.

FUN page is its own little ADT:

- **OP row:** SLEEP (rest at non-access vs skip), TRIG (gate width: match-clock vs short-trigger), SCAN (overwrite all 16 with current knob values).
- **MOD row:** RESET / CLK / RUNSTP / DIR — what an external gate to the channel's MOD input does.
- **CV row:** ADD (CV adds to current cell value, then quantise) / LOC (CV addresses cell directly) / SNAKE (CV selects snake pattern) / S&H (CV is sample-and-held on MOD rising edge).

Z-axis state machine:

- 64 states across 4 banks of 16. A state captures *all* per-channel programming including FUN/QUANT/CV.
- Z-MOD increments through enabled states; Z-CV addresses absolutely.
- Mesh-Paste page: enable multiple states for synchronised editing ("change one knob, change all enabled states").
- Multi-Paste: copy current state to all enabled states.
- Latch page: per-channel override (touch = always-on, regardless of ACCESS).

Select Bus: bidirectional state-sync with TEMPI and other Make Noise modules. Not relevant to a software port.

What's actually novel

The 64-state Z-axis is genuinely interesting as *hardware*: it lets the performer prep a session by authoring 16 variants per bank, then play the whole set as a real-time meta-instrument. But the *value* of this in performance comes from:

- Touching a state on the State Select page = instant change.
- Mesh-Paste lets you mass-edit while playing.
- The 64 stored variants give you a session-shaped catalogue at hand.

In a code-only context, these are all things we get for free: the "variant" is just another cell, the "mesh edit" is editing a shared PureScript module, the "instant change" is firing a different cell. **The Z-axis is essentially an inside-the-module version of the "verse/chorus/bridge" structuring problem** the calypso doc parks — we already have better answers in the works (cells, scenes, shared prelude).

What's left after subtracting Z-axis?

1. **Per-cell record** with (CV, gate, glide, access) — trivial.
2. **Snake pattern walk** through a 4×4 grid — trivial; just an index permutation.
3. **Cartesian channel** with two clocks and (col,row) lookup — this is the genuinely novel mechanic.
4. **Quantizer** per channel — we already have scales in Tidal.Cell.
5. **FUN page modes** — these are mostly hardware-physical (RESET, CLK-as-rising-edge, etc.); the SLEEP option ("rest at skipped cell" vs "skip") is the one real ADT distinction worth keeping.
6. **Latch page** — pure hardware-UX; no software analogue worth building.

The Cartesian channel is the find. **In mini-notation today, expressing "two clocks at different rates select cells from a 16-cell space" is awkward** — you'd need carefully-tuned <> alternation rates and it gets brittle fast. As a primitive, it's clean:

```
cartesian grid (xClk, yClk)
```

...where ```` cartesian grid (xClk, yClk)

```
...where `xClk` and `yClk` are independent Patterns of trigger times.
Their LCM defines the cycle; their interaction defines the texture.
A 4:3 ratio gives a 12-tick cycle through the 16 cells with a
characteristic non-Euclidean shape.
```

```
### Proposed PureScript surface (slim)
```

```
````purescript
module Tidal.Cartesian where

-- A single cell as René sees it
```

```

type Cell =
 { cv :: Number -- pre-quantise voltage 0..1
 , gate :: Boolean
 , glide :: Boolean
 , access :: Boolean -- if false, skip (or sleep, see below)
 }

-- 4x4 grid
newtype Grid = Grid (Vec4 (Vec4 Cell))

-- Snake patterns (the 16 hardcoded curves on the SNAKE page)
data SnakeCurve
 = LinearLR | LinearTB | Boustrophedon | SpiralIn | SpiralOut | ...
 -- 16 named curves total

-- A walking channel: snake-walks a grid via one clock
walkSnake :: Grid -> SnakeCurve -> Pattern Trigger -> Pattern Cell

-- The Cartesian channel: two clocks, (col,row) lookup
walkCartesian :: Grid -> Pattern Trigger -> Pattern Trigger -> Pattern Cell

-- Cell -> (Note, Gate, Glide) for downstream consumption
quantize :: Scale -> Pattern Cell -> Pattern (Note, Gate, Glide)

-- SLEEP mode: skipped cells produce silence-for-clock vs vanish entirely
data AccessMode = Skip | Sleep
withAccess :: AccessMode -> Pattern Cell -> Pattern Cell

```

That's it. **This is a small, principled extraction** of the parts of René that aren't hardware-UX-as-feature.

## Examples

```

-- 4x4 of notes (using a small builder)
g :: Grid
g = grid
 [[c4 , e4 , .gate g4 .glide, b4]
 , [d4 , skip f4, .gate a4 , c5]
 , [skip c4, e4 , .glide g4 , b4]
 , [d4 , .glide f4, skip a4 , c5]
]

-- Two-clock cartesian at 4:3, triplet vs quarter
melody :: Pattern Cell
melody = walkCartesian g (mini "1*4") (mini "1*3")
 # quantize cMinor

-- A snake walk that happens to be reverse-boustrophedon
snake1 :: Pattern Cell
snake1 = walkSnake g Boustrophedon (mini "1*16") # quantize dorian

```

## What we deliberately drop

- Z-axis state machine, mesh-paste, multi-paste, latch — replaced by cell-level versioning and the (TBD) scene-arming work.
- The Select Bus.
- The 16 knob CV programming UI — we type the values into the cell.
- The TEMPI follow-state-changes feature.

These removals mean the resulting "René primitive" is **maybe 5%** of the module's surface area but **arguably 80%** of its musical value in a code context. The extraction is clean.

## Estimated implementation cost

**Phase 1** (~3 hours): the Grid + Cell types + walkSnake + the 16 snake curves as data.

**Phase 2** (~3 hours): walkCartesian with two independent trigger patterns, including the (col, row) → cell lookup and rate-mismatch handling.

**Phase 3** (~1 hour): quantizer (we already have scales).

**Phase 4** (~2 hours): rig binding.

Total: ~9 hours, but **lower priority than Pam or Mimetic** because the gap it fills is narrower (Pam fills a clear hole; Mimetic gives a new gesture; René gives one more navigation primitive that's nice but not load-bearing).

## Module 4 — Intellijel Metropolis

**Verdict: don't port standalone — let it emerge from Pam.**

Andrew doesn't own a Metropolis. The previous research memo (six modules, 2026-04-25) flagged Metropolis as "the natural North Star for the language extension." After re-reading the manual and putting it next to the Pam analysis, that framing is the *correct* one — but the conclusion isn't "port Metropolis"; it's "build Pam and notice that you're already converging on Metropolis's shape."

## Hardware feature inventory

Metropolis is two of everything plus eight of one thing:

- **Master sequence:** 8 stages, each with a `````purescript` module `Tidal.Cartesian` where

```
-- A single cell as René sees it type Cell = { cv :: Number -- pre-quantise voltage 0..1 , gate :: Boolean ,
glide :: Boolean , access :: Boolean -- if false, skip (or sleep, see below) }
```

```
-- 4x4 grid newtype Grid = Grid (Vec4 (Vec4 Cell))
```

```
-- Snake patterns (the 16 hardcoded curves on the SNAKE page) data SnakeCurve = LinearLR | LinearTB |
Boustrophedon | SpiralIn | SpiralOut | ... -- 16 named curves total
```

```
-- A walking channel: snake-walks a grid via one clock walkSnake :: Grid -> SnakeCurve -> Pattern
Trigger -> Pattern Cell
```

-- The Cartesian channel: two clocks, (col,row) lookup walkCartesian :: Grid -> Pattern Trigger -> Pattern Trigger -> Pattern Cell

-- Cell → (Note, Gate, Glide) for downstream consumption quantize :: Scale -> Pattern Cell -> Pattern (Note, Gate, Glide)

-- SLEEP mode: skipped cells produce silence-for-clock vs vanish entirely data AccessMode = Skip | Sleep withAccess :: AccessMode -> Pattern Cell -> Pattern Cell

That's it. **\*\*This is a small, principled extraction\*\*** of the parts of René that aren't hardware-UX-as-feature.

### Examples

```purescript

-- 4x4 of notes (using a small builder)

g :: Grid

g = grid

```
[ [ c4  , e4  , .gate g4 .glide, b4 ]
  , [ d4  , skip f4, .gate a4      , c5 ]
  , [ skip c4, e4  , .glide g4      , b4 ]
  , [ d4  , .glide f4, skip a4      , c5 ]
  ]
```

-- Two-clock cartesian at 4:3, triplet vs quarter

melody :: Pattern Cell

melody = walkCartesian g (mini "1*4") (mini "1*3")

quantize cMinor

-- A snake walk that happens to be reverse-boustrophedon

snake1 :: Pattern Cell

snake1 = walkSnake g Boustrophedon (mini "1*16") # quantize dorian

What we deliberately drop

- Z-axis state machine, mesh-paste, multi-paste, latch — replaced by cell-level versioning and the (TBD) scene-arming work.
- The Select Bus.
- The 16 knob CV programming UI — we type the values into the cell.
- The TEMPI follow-state-changes feature.

These removals mean the resulting "René primitive" is **maybe 5%** of the module's surface area but **arguably 80%** of its musical value in a code context. The extraction is clean.

Estimated implementation cost

Phase 1 (~3 hours): the Grid + Cell types + walkSnake + the 16 snake curves as data.

Phase 2 (~3 hours): walkCartesian with two independent trigger patterns, including the (col, row) → cell lookup and rate-mismatch handling.

Phase 3 (~1 hour): quantizer (we already have scales).

Phase 4 (~2 hours): rig binding.

Total: ~9 hours, but **lower priority than Pam or Mimetic** because the gap it fills is narrower (Pam fills a clear hole; Mimetic gives a new gesture; René gives one more navigation primitive that's nice but not load-bearing).

Module 4 — IntelliJ Metropolis

Verdict: don't port standalone — let it emerge from Pam.

Andrew doesn't own a Metropolis. The previous research memo (six modules, 2026-04-25) flagged Metropolis as "the natural North Star for the language extension." After re-reading the manual and putting it next to the Pam analysis, that framing is the *correct* one — but the conclusion isn't "port Metropolis"; it's "build Pam and notice that you're already converging on Metropolis's shape."

Hardware feature inventory

Metropolis is two of everything plus eight of one thing:

- **Master sequence:** 8 stages, each with a (pitch slider, pulse count 1–8, gate type) triple. Drives **two tracks** (TRK1, TRK2) simultaneously.
- **Per-track playback parameters:** ORDER (19 modes — Linear, Reverse, PingPong, Random, Brownian, Address, etc.), LEN (1..8), DIV (clock divider), SWING, SLIDE TIME, gate length.
- **Per-track override lanes** (each lane is 8-step):
 - PITCH, GATE, RATCHet count, PROBability of playback, ACCUM (accumulating transposition), SLIDE, SKIP, CV.
- **8 MOD lanes** — each is itself an 8-step CV/gate sub-sequencer with its own ORDER, LEN, DIV. Routable to one of two assignable outputs or to dozens of internal destinations.
- **Mod matrix:** ~30 internal destinations (BPM, ORDER, LEN, swing, slide, probability, ratchet, gate length, pre/post pitch offset, octave, root, scale, accumulator settings, ...). Sources: 3 AUX inputs, 2 CTRL knobs, 8 MOD lanes. Each mapping is a triple (source, destination, target ∈ {TRK1, TRK2, TRK1+2}). **MOD lanes can target each other's ORDER/LEN/DIV** — full self-modulation, with one-tick-lag for cycles.
- **Per-stage gate type:** a 4-way ADT — HOLD | MULTIPLE | SINGLE | REST. HOLD stretches the gate across stage boundaries, which is *not* something mini-notation expresses cleanly.
- **Accumulator:** per-stage ACCUM lane folds over playback history (transposition that *accumulates* each time a stage plays, with a separately-specified reset rule).

Why this is the target shape, not the next port

Inspect each Metropolis feature against the Pam port plan:

| Metropolix feature | Pam analogue | Marginal cost |
|--|---|---------------|
| Per-stage record | Per-lane recipe | Already there |
| Closed-ADT gate type | Pam's Wave ADT | Already there |
| Per-track ORDER (19 modes) | Lane modifier + Euclidean | Partly there |
| ACCUM (history-folding) | Stateful overlay on recipe params | Need to add |
| Mod-matrix as triple | ParamMod (lane, param, combine, pat) | Already there |
| 8 MOD lanes | The 8 Pam lanes — same shape | Already there |
| Self-modulation (lane → lane) | ParamMod whose pat queries another | Need to add |
| ~30 destinations | Just Param enum + per-lane targets | Already there |
| Per-stage (pulse , gate_type , ...) | Need stage-level vocabulary above lane | Need to add |
| Two tracks fed by master | Need master/track distinction | Need to add |

The conclusion: **maybe 60% of Metropolix is already in the Pam port plan, just under different names**. The 40% that isn't — ACCUM, self-modulation as a runtime graph, master-vs-track hierarchy, per-stage gate-type ADT — should be **driven by use**, not built speculatively. Each addition should answer "I tried Pam and it couldn't do X." That's how you avoid the trap of porting an instrument's surface area without porting its *use*.

What's distinctive enough to flag now

Three Metropolix features deserve explicit memory even if we don't build them yet:

1. **Self-modulating MOD lanes**. Lane B's ORDER is a Pattern, and Lane A's CV is wired to Lane B's ORDER. Each tick: query Lane A, write its value to Lane B's ORDER for the *next* tick (one-tick-lag is necessary to break the loop). In our Pattern model this is just a **ParamMod** whose **pat** field is *another lane's pattern* — but it requires the runtime to resolve lanes in topological order with a one-tick-lag for cycles. **Worth a paragraph in Tidal.Pam's Phase 2 mod-matrix design**.
2. **Per-stage gate type as an ADT**. **HOLD** is the interesting one because it crosses event boundaries — a HOLD stage's gate stays high for as long as the *next* stage's pulse_count permits. This is genuinely hard to express in mini-notation without special-case combinator support. **A holds modifier on discrete events would be a useful Tidal primitive in its own right**.
3. **ACCUM with a reset rule**. ACCUM transposes a stage's pitch by **accum_amount × times_this_stage_has_played**, with reset rules like "every N cycles" or "on RESET trigger." This is a *fold* over playback history, not a function of position. Existing Tidal patterns are stateless query functions; ACCUM needs a wrapper that maintains state across queries. **Useful as a general-purpose accumulating combinator** — could see use far beyond Metropolix-style stage transposition.

Sketch surface (deferred — don't build until Pam earns it)

```

-- Metropolix-shaped extension on top of Tidal.Pam
type Stage =
  { pitch      :: Number      -- 0..1 normalised
  , pulses     :: Int         -- 1..8
  , gateType   :: GateType
  , ratchet    :: Int
  , prob       :: Number      -- 0..1
  , accumAmt   :: Number      -- transpose per repeat
  , slide      :: Boolean
  , skip       :: Boolean
  , cv         :: Number
  }

data GateType = Hold | Multiple | Single | Rest

type MasterSequence = Vec8 Stage

data Order = Linear | Reverse | PingPong | Random | Brownian | Address Int
           | ... -- 19 modes total

type Track =
  { order :: Order, len :: Int, div :: Int
  , swing :: Number, slideTime :: Number, gateLength :: Number
  , overrides :: TrackOverrides
  }

type TrackOverrides =
  { pitch :: Maybe (Vec8 Number)
  , gate  :: Maybe (Vec8 Boolean)
  , ratch :: Maybe (Vec8 Int)
  , prob  :: Maybe (Vec8 Number)
  , accum :: Maybe (Vec8 Number)
  , slide :: Maybe (Vec8 Boolean)
  , skip  :: Maybe (Vec8 Boolean)
  , cv    :: Maybe (Vec8 Number)
  }

-- The master/track/mod hierarchy
type Metropolix =
  { master :: MasterSequence
  , trk1    :: Track
  , trk2    :: Track
  , mods    :: Vec8 Pam.Lane -- the 8 MOD lanes are Pam lanes
  , matrix  :: Array MetroMod -- the explicit mod graph
  }

data Destination = DBpm | DOrder TrackId | DLen TrackId | ...
                 | DLaneOrder LaneIdx | DLaneLen LaneIdx | ...

type MetroMod =
  { source :: Source      -- Aux | Ctrl | Lane LaneIdx
  , dest   :: Destination
  }

```

```
, target :: TargetTrack    -- TRK1 | TRK2 | Both
, scale  :: Number
}
```

Don't build this now. Build Pam, get to where you'd reach for stage- level structure, and *then* see whether the extension wants to look like this or something simpler that emerged organically.

Estimated cost (if Metropolis is ever pursued directly)

~30-40 hours total, distributed:

- Master sequence + per-stage ADT: ~5 h
- Per-track overrides + 19 ORDER modes: ~6 h
- Mod-matrix-as-runtime-graph with topological resolution and one-tick-lag: ~10 h (this is the hard part)
- ACCUM with reset rules: ~4 h
- HOLD-style gate-type semantics on the event stream: ~5 h
- Integration with Pam's existing infrastructure: ~5 h
- Testing and rig binding: ~5 h

A non-trivial undertaking. **Recommended: defer until Pam has been in real use for a few months and concrete Metropolis-shaped wishes have been articulated.** Speculative Metropolis work would burn 30 hours that are better spent elsewhere.

Module 5 — Mutable Marbles (especially DEJA VU)

Verdict: port DEJA VU as a standalone Pattern combinator. Skip the rest.

Andrew owns Marbles. The whole module is rich — a clock generator (````purescript -- Metropolis-shaped extension on top of Tidal.Pam type Stage = { pitch :: Number -- 0..1 normalised , pulses :: Int -- 1..8 , gateType :: GateType , ratchet :: Int , prob :: Number -- 0..1 , accumAmt :: Number -- transpose per repeat , slide :: Boolean , skip :: Boolean , cv :: Number }

```
data GateType = Hold | Multiple | Single | Rest
```

```
type MasterSequence = Vec8 Stage
```

```
data Order = Linear | Reverse | PingPong | Random | Brownian | Address Int | ... -- 19 modes total
```

```
type Track = { order :: Order, len :: Int, div :: Int , swing :: Number, slideTime :: Number, gateLength :: Number , overrides :: TrackOverrides }
```

```
type TrackOverrides = { pitch :: Maybe (Vec8 Number) , gate :: Maybe (Vec8 Boolean) , ratch :: Maybe (Vec8 Int) , prob :: Maybe (Vec8 Number) , accum :: Maybe (Vec8 Number) , slide :: Maybe (Vec8 Boolean) , skip :: Maybe (Vec8 Boolean) , cv :: Maybe (Vec8 Number) }
```

```
-- The master/track/mod hierarchy type Metropolis = { master :: MasterSequence , trk1 :: Track , trk2 :: Track , mods :: Vec8 Pam.Lane -- the 8 MOD lanes are Pam lanes , matrix :: Array MetroMod -- the explicit mod graph }
```

```
data Destination = DBpm | DOrder TrackId | DLen TrackId | ... | DLaneOrder LaneIdx | DLaneLen LaneIdx |
```

```
...
```

```
type MetroMod = { source :: Source -- Aux | Ctrl | Lane LaneIdx , dest :: Destination , target :: TargetTrack
-- TRK1 | TRK2 | Both , scale :: Number }
```

Don't build this now. Build Pam, get to where you'd reach for stage-level structure, and *then* see whether the extension wants to look like this or something simpler that emerged organically.

Estimated cost (if Metropolis is ever pursued directly)

~30–40 hours total, distributed:

- Master sequence + per-stage ADT: ~5 h
- Per-track overrides + 19 ORDER modes: ~6 h
- Mod-matrix-as-runtime-graph with topological resolution and one-tick-lag: ~10 h (this is the hard part)
- ACCUM with reset rules: ~4 h
- HOLD-style gate-type semantics on the event stream: ~5 h
- Integration with Pam's existing infrastructure: ~5 h
- Testing and rig binding: ~5 h

A non-trivial undertaking. **Recommended:** defer until Pam has been in real use for a few months and concrete Metropolis-shaped wishes have been articulated. **Speculative Metropolis work would burn 30 hours that are better spent elsewhere.**

Module 5 – Mutable Marbles (especially DEJA VU)

Verdict: port DEJA VU as a standalone Pattern combinator. Skip the rest.

Andrew owns Marbles. The whole module is rich – a clock generator (`t1/t2/t3``), a random voltage generator (`X1/X2/X3``), a quantizer (STEPS), and the DEJA VU loop memory. The previous research memo already characterised these as separable pieces; this pass focuses on the one piece whose lift gives the highest live-coding return.

What DEJA VU actually does

Mechanics from the manual, distilled:

- The DEJA VU control is a single knob with two-stage semantics:
 - **7 → 12 o'clock** (`lockProb` 0..1`): probability of *replaying* a past sample instead of generating a fresh one. At 0, every sample is fresh (pure random); at 1, the loop is locked and nothing fresh is generated.
 - **12 → 5 o'clock** (`shuffleProb` 0..1`): once locked, probability of *jumping* to a random position within the loop instead of

reading sequentially. At 0, it's a pure repeating loop; at 1, it's a random permutation of the same set of samples.

- **Loop length**: 1 to 16 steps.
- **Apply scope**: rhythm only, voltages only, both, or neither.
- **The loop stores a seed, not voltages.** Quote: *"the 'loop' doesn't store the actual voltages, but a kind of 'seed' to generate them. While the sequence is looping, you can still apply transformations to it – like spreading the notes apart or shifting them up/down."*

That last point is the *crucial* design move. By storing a seed, the loop becomes:

- **Reproducible** across cells, sessions, and code edits.
- **Cheaply transformable** – change a downstream parameter (scale, spread, octave) and the same "shape" plays through it.
- **Hashable** (just the seed + length), so cells with identical DEJA VU configurations share state.

Why this is a great Pattern combinator

Mini-notation has `?` for per-event probability, but it has *no* way to say "make this random pattern repeatable." Once you've used `?`, the pattern is non-deterministic and untestable. DEJA VU addresses exactly that gap.

The shape:

```
```purescript
```

```
-- A combinator that wraps any Pattern whose query involves
-- randomness, replacing fresh draws with seed-driven draws that
-- are recoverable.
```

```
dejaVu
```

```
 :: forall a
 . { length :: Int
 , lockProb :: Number -- 0..1
 , shuffleProb :: Number -- 0..1
 , seed :: Int
 }
 -> Pattern a
```

```
-> Pattern a
```

```
-> Pattern a
```

Implementation idea: when the wrapped Pattern would call the runtime RNG, the wrapper instead consults a circular buffer of `length` "slots" indexed by step-number-mod-length. Each slot is the seed`

Implementation idea: when the wrapped Pattern would call the runtime RNG, the wrapper instead consults a circular buffer of `length` "slots" indexed by step-number-mod-length. Each slot is the seed`

for that step's draws (not the draws themselves). With probability ``(1 - lockProb)``, generate a fresh seed and write it into the slot; otherwise read the existing slot. Then with probability ``shuffleProb``, read a random other slot instead of the sequential one.

The seed-driven design means every wrapped Pattern's downstream transformations (``scale``, ``transpose``, ``quantize``) \*re-evaluate fresh\* against the same draws – so changing the scale of a DEJA-VU'd pattern is non-destructive, exactly as on hardware.

### ### Example cells

```
```purescript
-- A 5-step looped melody from a chromatic-random source
melody1 :: Pattern Note
melody1 = deJaVu { length: 5, lockProb: 0.9, shuffleProb: 0.0, seed: 42 }
               $ randomNotes (c2 .. c5)

-- The same loop but morph: locked but shuffling
melody2 :: Pattern Note
melody2 = deJaVu { length: 5, lockProb: 1.0, shuffleProb: 0.6, seed: 42 }
               $ randomNotes (c2 .. c5)

-- DEJA VU on rhythm: a 3-step looping pattern of probabilistic gates
hat :: Pattern Bool
hat = deJaVu { length: 3, lockProb: 0.95, shuffleProb: 0.0, seed: 17 }
        $ randomGates 0.4

-- Re-seed by changing seed in the cell – non-destructive: the
-- old version of the cell still reproduces its old loop deterministically.
melody3 :: Pattern Note
melody3 = deJaVu { length: 5, lockProb: 0.9, shuffleProb: 0.0, seed: 99 }
               $ randomNotes (c2 .. c5)
```

This addresses two things at once:

1. The "verse / chorus" structuring problem (each scene's seed gives a stable musical identity).
2. The "I want to bring back that beautiful random loop from yesterday" problem (commit the seed to git, recall it later).

What we don't port from the rest of Marbles

- **t-section jittery clock** — purerl-tidal already has tempo via link-spike; the t-section's "instrumentalist lagging and catching up" mechanic could be a *separate* ````purescript -- A 5-step looped melody from a chromatic-random source melody1 :: Pattern Note melody1 = deJaVu { length: 5, lockProb: 0.9, shuffleProb: 0.0, seed: 42 } $ randomNotes (c2 .. c5)`

```
-- The same loop but morph: locked but shuffling melody2 :: Pattern Note melody2 = deJaVu { length: 5, lockProb: 1.0, shuffleProb: 0.6, seed: 42 } $ randomNotes (c2 .. c5)
```

```
-- DEJA VU on rhythm: a 3-step looping pattern of probabilistic gates hat :: Pattern Bool hat = dejaVu {
length: 3, lockProb: 0.95, shuffleProb: 0.0, seed: 17 } $ randomGates 0.4

-- Re-seed by changing seed in the cell — non-destructive: the -- old version of the cell still reproduces
its old loop deterministically. melody3 :: Pattern Note melody3 = dejaVu { length: 5, lockProb: 0.9,
shuffleProb: 0.0, seed: 99 } $ randomNotes (c2 .. c5)
```

This addresses two things at once:

1. The "verse / chorus" structuring problem (each scene's seed gives a stable musical identity).
2. The "I want to bring back that beautiful random loop from yesterday" problem (commit the seed to git, recall it later).

What we don't port from the rest of Marbles

- **t-section jittery clock** — purerl-tidal already has tempo via link-spike; the t-section's "instrumentalist lagging and catching up" mechanic could be a **separate** `slop` combinator, but it's small enough that it doesn't need Marbles framing.
- **X-distribution** (constant / bell / uniform / discrete via SPREAD): a useful primitive, but it's "generate a random number with distribution X" — we don't need a whole module for that.
- **STEPS quantizer**: we already have scales in `Tidal.Cell`.
- **BIAS coin-flip clock split**: niche; if needed, a one-line combinator.

Estimated cost

Phase 1 (~3–4 hours): the `dejaVu` combinator with seed-driven lock/shuffle semantics. The hard part is ensuring downstream transformations (`scale`, etc.) re-evaluate fresh against the same seeds, which means the wrapping can't materialise values too early.

Phase 2 (~1–2 hours): per-cycle-or-per-event control over which parameters lock vs evolve, which mirrors the hardware's "DEJA VU applies to rhythm | voltages | both | neither" switch.

Total: **~5 hours**. Cheapest of the lot. Worth doing alongside the Pam port since both want the same seeded-RNG infrastructure.

Module 6 — Elektron Digitakt

Verdict: yes, port — possibly the strongest live-coding model of all six.

Andrew used to own a Digitakt and is familiar with the Elektron idiom. The Digitakt's design is unusual among the modules in this study because it was built for **performers programming patterns by hand on small step grids**, which means its data model is already organised

around "default behaviour plus sparse local overrides plus conditional behaviour." That's the same shape as a well-designed declarative cell.

Three orthogonal mechanisms

The Elektron sequencer model rests on three substantially independent ideas, each of which would be useful on its own and which together cover an extraordinary range of musical territory.

1. Parameter Locks ("P-locks")

Every step of every track can independently override **any** of the track's parameter values. The track has defaults (the values you set on the parameter pages); each step optionally specifies overrides that apply **only** when that step plays. A locked parameter on a single step doesn't affect the rest of the pattern.

Quote from the manual:

```
> "Up to 72 different parameters can be locked in a pattern. A
> parameter counts as one (1) locked parameter no matter how many
> trigs that lock it."
```

The mental model: the track is a function `Step -> Sound`, where most steps return `defaults` and a few steps return `defaults // overrides`. Compositionally:

```
trackOutput :: Step -> Sound
trackOutput step = defaults mergeWith Map.lookup step locks
```

This is **exactly** the right shape for live-coding cells. The default is what you author once; the locks are what you author gesturally.

In Digitakt practice, P-locks are the primary expressive mechanism between knob movements: instead of automating a filter cutoff, you P-lock the filter on specific steps. Sound design is structural ("on step 7 the snare gets pitched up"), not modulation-flavoured ("a slow LFO drives the filter").

2. Pattern pages

A pattern can have more than 16 steps – up to 64 in Digitakt I, more in II – laid out as **pages** of 16 steps each, **cycling**. Page 1 plays during the first bar, page 2 during the second, and so on, with the active page indicated on the front panel.

This solves the "long sequence from a small grid" problem: you don't need a 64-step display to author a 64-step pattern; you need a 16-step display and a page-switch.

```
In live-coding terms: **the outer pattern is ````
trackOutput :: Step -> Sound
trackOutput step =
  defaults `mergeWith` Map.lookup step locks
```

This is **exactly** the right shape for live-coding cells. The default is what you author once; the locks are what you author gesturally.

In Digitakt practice, P-locks are the primary expressive mechanism between knob movements: instead of automating a filter cutoff, you P-lock the filter on specific steps. Sound design is structural ("on step 7 the snare gets pitched up"), not modulation-flavoured ("a slow LFO drives the filter").

2. Pattern pages

A pattern can have more than 16 steps — up to 64 in Digitakt I, more in II — laid out as **pages of 16 steps each, cycling**. Page 1 plays during the first bar, page 2 during the second, and so on, with the active page indicated on the front panel.

This solves the "long sequence from a small grid" problem: you don't need a 64-step display to author a 64-step pattern; you need a 16-step display and a page-switch.

In live-coding terms: **the outer pattern is Pattern (Pattern Step)**. The outer cycles the pages; each inner is a 16-step bar. This composes with Pam-style mod-matrix overlays, with conditional trigs, with everything.

It also gives you a **frame for verse/chorus structuring at the track level**: a 64-step pattern with four pages is structurally "four 16-bar sections that play in sequence." That's the verse/chorus question Andrew parked, expressed as a single track-level composition rather than as a separate scene-arming layer.

3. Conditional trigs

Each trig can carry a **condition** — a boolean test that decides whether the trig fires *this time around*. The Digitakt vocabulary:

Condition	Fires when
FILL	Fill mode is active
PRE	The previous trig played
!PRE	The previous trig did NOT play
NEI	The neighbouring track's trig played
!NEI	The neighbouring track's trig did NOT play
1ST	First time this pattern plays
A:B	The A-th time out of every B (e.g. 1:4 = first of every 4 cycles)
X%	Random, with X% probability

This is genuinely distinctive. Mini-notation has **?** for probability, but it has no equivalent for **PRE**, **NEI**, **1ST**, or **A:B** — those are conditions on **pattern history** or **cross-track coupling**.

PRE is especially interesting: it gives you cause-and-effect between consecutive trigs, which is a richer-than-percent probability tool — you can build a "snare answers kick if kick played" relationship that's compositionally meaningful, not just statistically interesting.

A:B gives you long-period structure cheaply — **1:4** means "every fourth cycle," which is verse/chorus structure as a per-trig modifier.

NEI is cross-track coupling, same idea as Metropolix's mod matrix but at the trig level rather than the parameter level.

Why this might be the strongest model

Looking at the six modules together:

Module	Per-step variation	Long-period structure	Cross-track coupling	Live-friendly
Pam	Lane-level only	Loop param	Mod matrix	Yes
Mimetic	Per-step CV vector	None native	None native	Yes
René 2	Per-cell record	Z-axis states	None native	Hardware-UX
Metropolix	Per-stage record	Track ORDER	Mod matrix	Yes
Marbles	DEJA VU loop	Loop length	Distribution shapes	Yes
Digitakt	Sparse P-locks	Pattern pages	Conditional trigs	Yes

Digitakt is the only one where all three columns are first-class, explicitly-orthogonal concepts. **The P-lock model in particular generalises P-am's recipe + mod-matrix into a single sparse-overrides representation that's easier to reason about**, easier to author, and easier to version-control. It's also conceptually unified with how a code editor works (defaults + diffs).

Proposed PureScript surface

```
module Tidal.Digitakt where

-- A parameter for an audio/MIDI track. Closed for now; we expand as
-- needed. This is the closed ADT that defines what's lockable.
data Param
  -- Pitch
  = Note          -- MIDI note (Int)
  | Velocity      -- 0..127
  | NoteLength   -- 0..1 of step time (0 = trig only)
  -- Sample (for audio tracks)
  | Sample       -- Int sample slot
  | StartPoint   -- 0..1 sample position
  | Length       -- 0..1 sample length
  -- Filter
  | FilterCut | FilterRes | FilterType
```

```

-- Amp
| AmpAttack | AmpHold | AmpRelease | AmpVolume | AmpPan
-- LFO
| LfoSpeed | LfoMul | LfoFade | LfoDest | LfoWave | LfoStart | LfoMode |
LfoDepth
-- (full set kept short here; in practice ~50 params per track)

-- A value for a parameter. Use Tidal.Pattern.Types.Value.
type ParamValue = Value -- VInt | VNumber | VBool | VString

-- The conditional-trig vocabulary
data TrigCondition
= Always
| Fill
| Pre Boolean -- True for PRE, False for !PRE
| Nei TrackIdx Boolean
| First | NotFirst
| EveryNthOfM Int Int -- A:B: the A-th time out of every B
| Probability Number -- 0..1

-- One step's overrides
type Step =
{ locks :: Map Param ParamValue -- P-locks (sparse)
, cond :: TrigCondition -- defaults to Always
, microT :: Number -- -0.5..+0.5 of step time
, retrig :: Maybe Int -- N micro-trigs within this step
}

defaultStep :: Step
defaultStep = { locks: Map.empty, cond: Always, microT: 0.0, retrig:
Nothing }

-- A pattern page: 16 steps (Maybe - Nothing means "no trig at this step")
type Page = Vec16 (Maybe Step)

-- A track: defaults plus an array of pages played in cycle
type Track =
{ defaults :: Map Param ParamValue -- the parameter-page values
, pages :: Array Page -- 1..N pages; N=1 is normal 16-
step
, length :: Int -- 1..16 (per-track length, can be
< 16)
}

-- A pattern: 8 tracks (audio + MIDI mixed, like the hardware)
type Digitakt =
{ tracks :: Array Track -- typically 8
, bpm :: Number
}

-- Run a Digitakt configuration
runDigitakt :: Digitakt -> Pattern (Map TrackIdx (Map Param ParamValue))

```

Example cells

```

-- A simple kick track: trigs on 1, 5, 9, 13. Step 9 has a
-- pitched-up p-lock; step 13 only fires every 4th cycle.
kick :: Track
kick =
  { defaults: Map.fromFoldable
    [ Sample /\ vint 0
    , Note /\ vnote c2
    , AmpVolume /\ vnum 0.9
    ]
  , pages:
    [ page0f
      [ Just defaultStep          -- step 1: standard kick
      , Nothing, Nothing, Nothing
      , Just defaultStep          -- step 5
      , Nothing, Nothing, Nothing
      , Just (defaultStep
        { locks = Map.fromFoldable
          [ Note /\ vnote c3 ] -- step 9: pitched up
        })
      , Nothing, Nothing, Nothing
      , Just (defaultStep
        { cond = EveryNthOfM 1 4 }) -- step 13: every 4th cycle only
      , Nothing, Nothing, Nothing
      ]
    ]
  , length: 16
  }

-- A 64-step pattern with four pages (verse, pre-chorus, chorus, bridge)
fullSong :: Track
fullSong = (defaultTrack hat)
  { pages = [ verse, preChorus, chorus, bridge ]
  , length = 16
  }

-- Conditional trig example: snare fires only if kick fires (track 0)
-- on the same step
snare :: Track
snare = kick
  { defaults = Map.insert Sample (vint 1) kick.defaults -- snare sample
  , pages = [ page0f
    [ Nothing
    , Nothing
    , Just (defaultStep { cond = Nei 0 true }) -- only if kick fired
    , ""
    ]
  ]
  }

```

What's gained, what's lost

Gained:

- The simplest possible authoring shape: defaults, with overrides where you want them. No mod-matrix DAG, no lane recipe records, no separate parameter ADT for each module.
- Pattern-pages are a clean answer to long-period structure that composes with everything else without needing a separate scene-arming concept.
- Conditional trigs are a *small* primitive vocabulary (8 conditions) that cover an enormous range of musical relationships. Worth having even if we never build the full Digitakt model — the

module Tidal.Digitakt where

-- A parameter for an audio/MIDI track. Closed for now; we expand as -- needed. This is the closed ADT that defines what's lockable. data Param -- Pitch = Note -- MIDI note (Int) | Velocity -- 0..127 | NoteLength -- 0..1 of step time (0 = trig only) -- Sample (for audio tracks) | Sample -- Int sample slot | StartPoint -- 0..1 sample position | Length -- 0..1 sample length -- Filter | FilterCut | FilterRes | FilterType - - Amp | AmpAttack | AmpHold | AmpRelease | AmpVolume | AmpPan -- LFO | LfoSpeed | LfoMul | LfoFade | LfoDest | LfoWave | LfoStart | LfoMode | LfoDepth -- (full set kept short here; in practice ~50 params per track)

-- A value for a parameter. Use Tidal.Pattern.Types.Value. type ParamValue = Value -- VInt | VNumber | VBool | VString

-- The conditional-trig vocabulary data TrigCondition = Always | Fill | Pre Boolean -- True for PRE, False for !PRE | Nei TrackIdx Boolean | First | NotFirst | EveryNthOfM Int Int -- A:B: the A-th time out of every B | Probability Number -- 0..1

-- One step's overrides type Step = { locks :: Map Param ParamValue -- P-locks (sparse) , cond :: TrigCondition -- defaults to Always , microT :: Number -- -0.5..+0.5 of step time , retrigger :: Maybe Int -- N micro-trigs within this step }

defaultStep :: Step defaultStep = { locks: Map.empty, cond: Always, microT: 0.0, retrigger: Nothing }

-- A pattern page: 16 steps (Maybe — Nothing means "no trig at this step") type Page = Vec16 (Maybe Step)

-- A track: defaults plus an array of pages played in cycle type Track = { defaults :: Map Param ParamValue -- the parameter-page values , pages :: Array Page -- 1..N pages; N=1 is normal 16-step , length :: Int -- 1..16 (per-track length, can be < 16) }

-- A pattern: 8 tracks (audio + MIDI mixed, like the hardware) type Digitakt = { tracks :: Array Track -- typically 8 , bpm :: Number }

-- Run a Digitakt configuration runDigitakt :: Digitakt -> Pattern (Map TrackIdx (Map Param ParamValue))

Example cells

```

``purescript
-- A simple kick track: trigs on 1, 5, 9, 13. Step 9 has a
-- pitched-up p-lock; step 13 only fires every 4th cycle.
kick :: Track
kick =
  { defaults: Map.fromFoldable
    [ Sample /\ vint 0
    , Note /\ vnote c2
    , AmpVolume /\ vnum 0.9
    ]
  , pages:
    [ pageOf
      [ Just defaultStep          -- step 1: standard kick
      , Nothing, Nothing, Nothing
      , Just defaultStep          -- step 5
      , Nothing, Nothing, Nothing
      , Just (defaultStep
        { locks = Map.fromFoldable
          [ Note /\ vnote c3 ] -- step 9: pitched up
        })
      , Nothing, Nothing, Nothing
      , Just (defaultStep
        { cond = EveryNthOfM 1 4 }) -- step 13: every 4th cycle only
      , Nothing, Nothing, Nothing
      ]
    ]
  , length: 16
  }

-- A 64-step pattern with four pages (verse, pre-chorus, chorus, bridge)
fullSong :: Track
fullSong = (defaultTrack hat)
  { pages = [ verse, preChorus, chorus, bridge ]
  , length = 16
  }

-- Conditional trig example: snare fires only if kick fires (track 0)
-- on the same step
snare :: Track
snare = kick
  { defaults = Map.insert Sample (vint 1) kick.defaults -- snare sample
  , pages = [ pageOf
    [ Nothing
    , Nothing
    , Just (defaultStep { cond = Nei 0 true }) -- only if kick fired
    , ""
    ]
  ]
  }

```

What's gained, what's lost

Gained:

- The simplest possible authoring shape: defaults, with overrides where you want them. No mod-matrix DAG, no lane recipe records, no separate parameter ADT for each module.
- Pattern-pages are a clean answer to long-period structure that composes with everything else without needing a separate scene-arming concept.
- Conditional trigs are a *small* primitive vocabulary (8 conditions) that cover an enormous range of musical relationships. Worth having even if we never build the full Digitakt model — the **EveryNth0fM** and **Pre/Nei** conditions in particular are generally useful.

Lost:

- The hardware Digitakt's tactile workflow (turn the knob to enter a P-lock value live). We replace this with cell editing, which is slower but version-controlled.
- Sound packs and the sample browser — irrelevant; we use the rig.
- Live-recorded automation. Could be added later as a "lane override" overlay.

Estimated cost

Phase 1 (~5-6 hours): Track, Step, Page, Param ADT; runDigitakt producing per-track event streams; integration with Tidal.Pattern.

Phase 2 (~3-4 hours): Conditional trig evaluation, including the cross-track NEI lookup (which needs a two-pass evaluator — first pass collects "did this step play?" per track per step, second pass evaluates conditions).

Phase 3 (~3 hours): Pattern-page rotation; per-track length that may differ from page length (creating cross-bar polyrhythms).

Phase 4 (~3 hours): Rig binding — mapping Track parameters to specific CV/gate destinations.

Total: **~14 hours** to a working Digitakt-style cell vocabulary.

Where this leaves Pam

Pam and Digitakt are **complementary, not redundant**. Pam excels at *continuous* shaped modulation (the triangle-LFO-at-/4 modulating a filter); Digitakt excels at *discrete* per-step variation (the filter-cutoff that's specifically different on step 7).

If we built only one, Digitakt would cover more musical ground for typical live-coding gestures. But built together, they form a clean two-axis system:

- **Continuous, shaped, parametric** → **Tidal.Pam**
- **Discrete, sparse, conditional** → **Tidal.Digitakt**

Both share underlying infrastructure (cv-router routing, voice supervisors, Pattern evaluation), and the cells using them can coexist in the same session.

Revised recommendation: build a *minimal* Digitakt (P-locks + default-or-override only, no pages, no conditionals) before Pam, then build Pam, then add conditional trigs and pages back into Digitakt. The P-lock primitive is the most foundational and unblocks the biggest expressive jump for the least code.

Honourable mentions

Brief notes on two more modules whose *shape* is worth being aware of, but where porting would mostly duplicate work already covered by the five modules above.

Acid Rain Maestro

Six clocked phasor-LFOs with **chain mode**: each channel cycles through a *list* of (waveform, length) pairs, giving the channel a state machine of looped LFO shapes that advances per-channel.

This is essentially **Pam plus a per-lane state machine**. The chain-state-machine idea isn't covered by Pam's recipe model — but it could be added as a `Lane.chain :: Maybe (Array LaneRecipe)` field that cycles each Loop reset. About 2 hours of work on top of Pam, if it earns its keep.

The specific Maestro detail worth flagging: lengths are in **clock pulses**, not seconds. So a 16-pulse triangle at /4 BPM takes 16 beats to complete one cycle. This "musically-quantised LFO" framing is already implicit in Pam's Modifier (which is beat-relative); chain mode just lets you sequence different shapes through it.

Five12 Vector Sequencer

Multi-part hardware sequencer. Two distinctive features the others don't fully cover:

1. **Per-step "type"**: each step carries a tag — `gate`, `hold`, `repeat`, `slide`, `rest`, `tie`. This is similar to Metropolis's per-stage gate-type ADT, but more elaborate (six types, each with its own continuation semantics for the next step).
2. **Cross-part modulation**: any part can read from any other part's CV at the current step. Like Metropolis's lane-targets- lane, but exposed at the part-not-mod level.

Both are subsumed by what we'd build for Metropolis if we ever chose to. **Don't port; flag the per-step-type ADT as inspiration when designing the gate-type extension to Pam.**

Why nothing else makes the list

Other candidates considered:

- **WMD Metron** (drum trigger sequencer with probability): subsumed by Pam.
- **Industrial Music Electronics Stillson Hammer mkII** (4-track CV+gate, per-step CV-input attenuation): subsumed by Pam plus the Metropolis-style per-track override lanes; doesn't add a novel mechanic.
- **WMD/SSF Toolbox, Doepfer A-154/A-155, Korg SQ-64**: variations on classic step sequencer themes; nothing not already covered.
- **Squarp Pyramid/Hapax**: pattern-of-patterns and polyphonic routing, but their value is largely in their UI/MIDI integration, not in algorithmic novelty.

The **distinctive primitives** worth knowing about (collected across both this and the prior research memo):

1. Pam-style typed lane recipe (covered).
2. Mimetic-style 5-verb nav action (covered).
3. René-style two-clock Cartesian walk (covered).

4. Metropolis-style mod-matrix with self-modulation and per-stage ADT (deferred).
5. Marbles-style DEJA VU seeded random loop (covered).
6. Maestro-style per-lane chain-state-machine (small extension to Pam if needed).
7. Marbles-style STEPS continuous quantiser (already have scales; the continuous-knob axis is a "future quantize-strength" param).
8. Ochd-style curated-irrational-ratios LFO bank (data, not code; add as a preset).

Cross-cutting observations

The shared kernel

All six modules instantiate the same abstract shape:

(StateStore, Navigator, Reader)

Module	StateStore	Navigator	Reader
Pam	8 lane recipes	clock tick	8 voltages
Mimetic	16 4-vectors	5-verb action	4-vector
René 2	4×4 grid × 3 channels (× 64 Z-states)	2-clock cartesian + access mask	(cv, gate, glide) × 3
Metropolis	master 8-stage + 2 tracks + 8 mod-lanes + matrix	per-track ORDER + clock	tracks' CV + gate + 2 assignable mod outs
Marbles	DEJA VU loop (seed buffer)	sequential read with lock/jump	random voltage / gate
Digitakt	defaults + sparse step overrides + pages	clock tick + condition test	per-track param map

They differ in *what* is stored and *how* the navigator can move through it. The richness comes from the navigator's vocabulary. The hardware reality of "navigator = patched gate" caps that vocabulary at the count of patch cables you have. **In code, vocabulary is unbounded.**

A second observation, after the Digitakt addition

The Digitakt model reveals a subtler insight: **the most live-coding- friendly state-stores are the ones that are already "defaults plus sparse local overrides."** That shape:

- Reads cleanly: most of the structure is the default; the deviations stand out.
- Edits cheaply: a per-step override is a one-key Map insertion.
- Diffs well: git diff on a cell with sparse overrides is immediately legible (it's just a new line in the locks map).
- Composes orthogonally: locks, conditions, pages, micro-timing are independent axes that can be added or removed individually.

Pam's recipe model is also "defaults plus a small mod-matrix," which fits the same template. Metropolis's per-stage record + override lanes is the same idea, scaled up. **The shape of a good live-coding**

sequencer is the shape of a good sparse data structure.

The inversion

This is the single most important observation:

Hardware sequencers have rich state-stores but impoverished navigators. In a live-coding language, we can preserve the state- store designs (which are the genuine art) while replacing navigators with Patterns of typed actions (which are strictly more expressive).

Once you see this, you stop thinking "should I port this module?" and start thinking "what is the minimal navigation vocabulary that makes this state-store musically alive?" For Pam, the navigator is just "clock tick" — but the param Pattern overlay is the navigator-of- recipe-fields, and that's where the language extension lives. For Mimetic, the navigator is genuinely a 5-verb action set. For René, the navigator is the two-clock cartesian — small but distinctive.

What this implies for live-coding semantics

The big win is **composability across modules**. In hardware, a Pam output and a Mimetic CV channel and a René C output are three different cables that you patch into three different things. In code, they're all **Pattern Voltage** (or richer types) — and we can:

- Cross-modulate (Pam's level driven by Mimetic's CV3).
- Multiplex (alternate banks per voice via **voiced**).
- Branch (jux a sequencer's output, layer reversed and reconverged).
- Quantise outputs of any of them to any scale.
- Wrap with **every, slow, fast, rev, mask**.

None of which is possible in modular without dedicated utility modules per cross-modulation — at which point you've spent more HP on plumbing than on instruments.

What we *can't* easily replicate

Touch interaction. Latch pages. Combo moves. Mesh-Paste's "simultaneous-edit-of-many-stored-variants." These are not deficiencies in the language; they're **different idioms**. Code- based versions are slower per-edit but composable, version-controlled, and shareable. Trade-off, not a loss.

Recommended sequencing

If we were to actually do any of this:

0. Minimal Digitakt-style first (~5-6 hours)

Build the Track + Step + Page + Param + P-locks core only. No conditional trigs yet, no pattern pages yet. Just default-and- overrides per step, integrated with Tidal.Pattern. **This is the foundational primitive** — once this is in, every other model becomes either an addition to it (conditional trigs, pages) or a sibling that shares its infrastructure (Pam, Mimetic). A surprising amount of musical ground opens up at this point.

0a. DEJA VU combinator alongside (~5 hours)

Tiny scope, big return, orthogonal to the Digitakt P-lock work. Shares seeded-RNG infrastructure with Pam-when-it-comes. Worth doing as a side-project alongside (0).

1. Add conditional trigs to Digitakt (~3-4 hours)

The Pre / Nei / 1ST / A:B / X% vocabulary. This is the second- most-impactful step after P-locks and unlocks long-period structure and cross-track coupling. Particularly worth pulling forward if verse/chorus structuring is a near-term concern (which the calypso scenes-and-shared-state doc suggests it is).

2. Add pattern pages to Digitakt (~3 hours)

Long sequences from short pages — this is the verse/chorus answer at the *track* level (rather than scene-arming at the cell level). Both layers can coexist; they answer the same question at different granularities.

3. Pam (~10 hours)

Pam delivers the typed *continuous* mod-matrix that complements Digitakt's *discrete* P-locks. After Digitakt is in real use, Pam fills the obvious gap: per-channel shaped continuous modulation. Sketch first phase:

- **Tidal.Pam** module with Lane, Wave, Modifier, Euclid, Pam types.
- **runPam :: Pam -> Pattern (Vec8 Voltage)** against the existing Pattern infrastructure.
- A handful of factory cells (4-on-the-floor, Euclidean hat, slow triangle LFO).
- **bind-pam** directive in tidal-cli for rig routing.

4. Mimetic (~6 hours)

A **third gesture axis** alongside Digitakt (sparse step overrides) and Pam (continuous shaped modulation): bank navigation by typed verb pattern. Smaller scope, complementary to both.

5. René's Cartesian primitive (~9 hours, optional)

Land the two-clock cartesian + snake walker, drop everything else. Treat it as a *primitive*, not a module. Resist the temptation to port the Z-axis; the Digitakt + scenes-and-shared-state combination already covers what made the Z-axis valuable.

6. Things to *not* do

- Don't port René's Z-axis as a runtime state machine; cells already give us this idea, better.
- Don't try to replicate Pam's flash banks; git is the answer.
- Don't replicate the touch UX of any of these in calypso — different medium, different gestures.
- **Don't build Metropolis as a separate project.** Build Pam, use it, and let Metropolis-shaped extensions emerge organically when the gaps become concrete. Speculative Metropolis work could easily burn 30+ hours on capability you may never reach for.
- Don't port the t-section, X-distribution, or STEPS from Marbles; they're individually small but cumulatively a distraction from the one Marbles idea (DEJA VU) that genuinely earns a slot.

Honest answer to "is this attractive enough to do soon?"

Digitakt-style P-locks: yes, soon — possibly the highest-leverage work item across all six modules.

The default-plus-overrides shape is so directly aligned with how cells should be authored that it's arguably *the* missing primitive in the cell vocabulary. ~5-6 hours of work for the foundational version.

DEJA VU combinator: yes, alongside Digitakt. Cheap, distinct, fills a real gap (no recoverable randomness in mini-notation today).

Pam: yes, after Digitakt is in use. Adds the continuous-shaped modulation axis that Digitakt's discrete-overrides axis doesn't cover. ~10 hours.

Mimetic: yes, opportunistically. Genuinely new gesture; modest scope; can land at any point after Digitakt.

René's Cartesian: optional. The two-clock polyrhythmic walker is interesting but not load-bearing once we have the other three axes covered.

Metropolix: not as a port. Build the things on the path to Metropolix; let the destination emerge.

Open questions

- 1. Mod-matrix semantics for Pam.** When a `Pattern Number` is bound to `lane.level`, does it (a) replace the static value entirely, (b) multiply it, or (c) add to it? The hardware does (a) replacement with attenuverter scaling. (a) is simplest; (b) and (c) are richer. Probably worth supporting all three explicitly: `replaceWith`, `scaleBy`, `addTo`.
- 2. Mimetic bank arity.** Strict 16-step (matching hardware) or variable-length? Variable is more general but less aligned with the 4x4 navigator semantics. Suggest: keep 16 by default, allow variable as `mimeticN :: Int -> Bank -> Pattern Nav -> ...`.
- 3. René Cartesian and access masks.** When neither X nor Y has advanced (because their clock patterns don't tick this event), what does the C channel emit? Hardware: holds last value. Software: either no event, or a "hold" event. Probably no event.
- 4. Glide as a property of cells, or a property of routing?** René ties glide to the cell. We could either (a) emit glide as part of the event (`{ note, gate, glideMs }`) and have the per-bus router handle it, or (b) tie glide to the bus configuration (`bind-cv ... --slew 25ms`). (a) is more local; (b) is closer to hardware. Probably (a) — keep the cell as the source of truth.
- 5. Where does this work live?** `Tidal.Pam`, `Tidal.Mimetic`, `Tidal.Cartesian` as siblings of `Tidal.Cell.Prelude`, all re-exportable into a future `Tidal.Sequencers` umbrella. None of them belong in `Tidal.Cell.Prelude` itself (too domain-specific for the default-in-scope set).
- 6. Daemon-path implications.** Each of these adds modules to the purerl-`tidal` compile graph. That bumps the per-cell compile time slightly (each cell that imports `Tidal.Pam` pulls Pam through the purs-backend-erl re-emit). PR4's daemon path is the answer; until then, larger language surface = slightly slower cells.
- 7. Random-seed determinism for Pam's Loop.** The semantics of "re-seeds every N beats" require a reproducible RNG keyed by (loop-index, lane-index, beat-index). Different RNG choices give different musical feels. Worth a small exploration of seeding strategies once we get there.

Pickup notes for a future session

If picking this up cold:

1. **Start with Pam Phase 1:** types + runPam + a single example cell driving 4 buses (kick, hat, lfo, env). That alone is a milestone.
2. **Then mod-matrix overlay:** pick one Pattern-driven param (probably level) and prove the loop closes.
3. **Mimetic comes second** and is quicker; both could land in a single week of focused work.
4. **René's Cartesian primitive** is on the back-burner unless Andrew finds himself reaching for two-clock polyrhythm and missing it.

The primary risk is **scope creep**: Pam in particular has 11 per-lane parameters and 26 banks worth of state — easy to drift into trying to replicate every behaviour. **The core deliverable is the typed schema + one-cycle Pattern emission**; everything else can come later if it earns its keep.

A secondary risk is **mod-matrix complexity**: the param-overlay mechanism, if implemented carelessly, becomes its own little metaprogramming language. Keep it small — one Pattern per (lane, param) destination, with a single combine-op (replace by default).

Appendix: rig wiring sketch for each port

A concrete picture of what goes where, in case it's useful for the "do this for real" pass.

Pam → rig

8 outputs, one per lane:

```
lane[0] gate    → ES-5 gate 1    (kick trigger to synth/sampler)
lane[1] gate    → ES-5 gate 2    (snare trigger)
lane[2] gate    → ES-5 gate 3    (hat trigger)
lane[3] tri LFO → ES-9 bus 8     (panel jack 1 → filter cutoff CV)
lane[4] env     → ES-9 bus 9     (panel jack 2 → VCA CV)
lane[5] sine    → ESX-8CV out 1  (vibrato to oscillator)
lane[6] random  → ESX-8CV out 2  (sample-and-hold style mod)
lane[7] gate    → FH-2 gate      (MIDI synth trigger via FH-2)
```

Eight different outputs into eight different fates is exactly what Pam is for. The cell-level cost: one bind statement per lane; the runtime cost: one event scheduler per lane.

Mimetic → rig

4 CVs from the bank, plus a per-step trigger:

```
voice[0] → ES-9 bus 8 (panel jack 1) // pitch CV to VCO 1
voice[1] → ES-9 bus 9 (panel jack 2) // filter cutoff
voice[2] → ESX-8CV out 1              // resonance
```

```
voice[3] → ESX-8CV out 2 // amp envelope amount
trigger → ES-5 gate 1 // step trigger
```

Or, more interestingly: route the four CVs all to the *same* synth voice, controlling 4 different parameters of one timbre. That's the "4-vector per step" interpretation that makes Mimetic distinctive.

René Cartesian → rig

Three channels, each with CV + gate:

```
X channel CV → ES-9 bus 8 // pitch
X channel gate → ES-5 gate 1 // trigger
Y channel CV → ES-9 bus 9 // pitch
Y channel gate → ES-5 gate 2 // trigger
C channel CV → ESX-8CV out 1 // pitch (the cartesian-derived voice)
C channel gate → ES-5 gate 3 // trigger
```

Three voices, two independent and one cross-modulated by both. The classic René patch.

*Document complete. Three modules examined; one strong port recommendation (Pam), one good port recommendation (Mimetic), one selective extraction (René Cartesian). The work, if pursued, totals ~25 hours and produces three concrete language extensions to purerl-tidal. None are blocking; all sit cleanly in the

```
lane[0] gate → ES-5 gate 1 (kick trigger to synth/sampler)
lane[1] gate → ES-5 gate 2 (snare trigger)
lane[2] gate → ES-5 gate 3 (hat trigger)
lane[3] tri LFO → ES-9 bus 8 (panel jack 1 → filter cutoff CV)
lane[4] env → ES-9 bus 9 (panel jack 2 → VCA CV)
lane[5] sine → ESX-8CV out 1 (vibrato to oscillator)
lane[6] random → ESX-8CV out 2 (sample-and-hold style mod)
lane[7] gate → FH-2 gate (MIDI synth trigger via FH-2)
```
```

Eight different outputs into eight different fates is exactly what Pam is for. The cell-level cost: one bind statement per lane; the runtime cost: one event scheduler per lane.

### Mimetic → rig

4 CVs from the bank, plus a per-step trigger:

```
` ``
voice[0] → ES-9 bus 8 (panel jack 1) // pitch CV to VCO 1
voice[1] → ES-9 bus 9 (panel jack 2) // filter cutoff
voice[2] → ESX-8CV out 1 // resonance
voice[3] → ESX-8CV out 2 // amp envelope amount
trigger → ES-5 gate 1 // step trigger
```

```
```
```

Or, more interestingly: route the four CVs all to the **same** synth voice, controlling 4 different parameters of one timbre. That's the "4-vector per step" interpretation that makes Mimetic distinctive.

René Cartesian → rig

Three channels, each with CV + gate:

```
```
```

```
X channel CV → ES-9 bus 8 // pitch
X channel gate → ES-5 gate 1 // trigger
Y channel CV → ES-9 bus 9 // pitch
Y channel gate → ES-5 gate 2 // trigger
C channel CV → ESX-8CV out 1 // pitch (the cartesian-derived voice)
C channel gate → ES-5 gate 3 // trigger
```
```

Three voices, two independent and one cross-modulated by both. The classic René patch.

```
---
```

Document complete. Three modules examined; one strong port recommendation (Pam), one good port recommendation (Mimetic), one selective extraction (René Cartesian). The work, if pursued, totals ~25 hours and produces three concrete language extensions to purerl-tidal. None are blocking; all sit cleanly in the `Tidal.Sequencers` umbrella alongside the existing `Tidal.Cell.Prelude`.